

Arquitecturación del Generador de Analizadores Sintácticos

Video de presentación: https://youtu.be/AL54rhos_gQ

Repositorio: <https://github.com/lfmendoza/compiladores-yapar>

1. Alcance y enfoque

El sistema completo se compone de dos generadores acoplados: el analizador léxico generado por **YALex** (entregado en la fase previa, basado en construcción de Thompson, subset construction y reconocimiento por DFA con regla de máximo prefijo) y el analizador sintáctico generado por **YAPar**, que es el objeto de esta arquitectura. YAPar lee una especificación de gramática libre de contexto, construye el autómata LR(0) canónico, deriva la tabla SLR(1) y produce un parser por pila que consume los tokens emitidos por el lexer. Antes de programar, se delimitan los módulos, sus contratos de entrada/salida y la representación interna del autómata, con el fin de que cada pieza pueda implementarse y probarse de manera independiente.

2. Módulos a programar

Cada módulo se concibe como una unidad con responsabilidad única, con una entrada, una salida explícita y sin acoplamiento con los detalles internos de los demás. La frontera entre módulos coincide con una estructura de datos serializable, lo que facilita las pruebas y la depuración a partir de archivos intermedios.

1. Lector de archivos YAPar (.yalp)

Parsea la especificación de gramática (declaraciones de tokens, IGNORE y producciones). **Entrada:** archivo .yalp. **Salida:** objeto Grammar con producciones, terminales declarados y símbolo inicial. Reporta errores sintácticos del propio archivo de gramática.

2. Analizador léxico (YALex, fase previa)

Reutiliza el DFA generado por YALex. **Entrada:** archivo fuente a parsear. **Salida:** flujo de tokens (tipo, lexema, línea, columna). Es externo al pipeline de YAPar pero su salida alimenta al motor del parser.

3. Representación interna de la gramática

Estructuras Symbol, Production y Grammar. Realiza la *augmentación* con la producción $S' \rightarrow S$ y clasifica símbolos en terminales/no terminales. **Entrada:** objeto crudo del lector. **Salida:** gramática aumentada lista para análisis.

4. Calculador de FIRST y FOLLOW

Algoritmo de punto fijo clásico. **Entrada:** gramática aumentada. **Salida:** dos diccionarios FIRST: NoTerm $\rightarrow$ Set(Term + {epsilon}) y FOLLOW: NoTerm $\rightarrow$ Set(Term + {\$}). FOLLOW es la pieza necesaria para SLR.

5. Constructor del autómata LR(0)

Implementa `closure(I)` y `goto(I, X)` y produce la colección canónica de conjuntos de items LR(0). **Entrada:** gramática aumentada. **Salida:** grafo de estados $\{I_0, I_1, \dots, I_n\}$ con transiciones etiquetadas por símbolos.

6. Generador de la tabla SLR(1)

Combina el autómata con FOLLOW para llenar las tablas ACTION y GOTO. Detecta y reporta conflictos shift/reduce y reduce/reduce. **Entrada:** autómata LR(0) y FOLLOW. **Salida:** tabla bidimensional indexada por (estado, símbolo).

7. Motor del parser (driver LR)

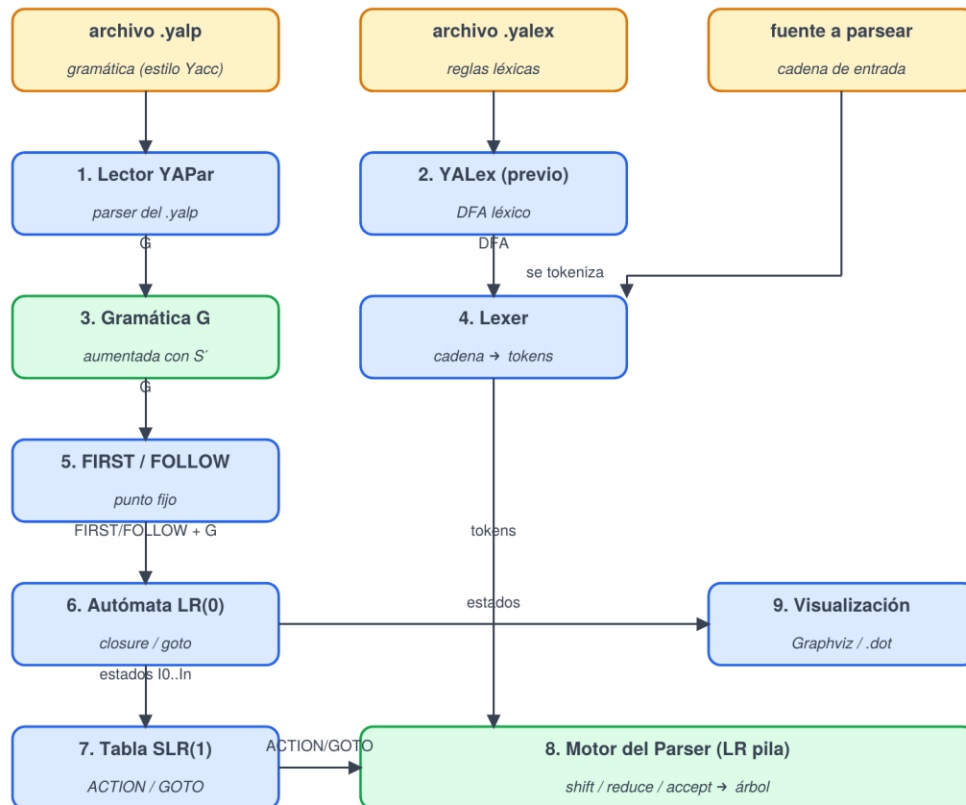
Algoritmo de pila estándar: lee un token, consulta ACTION[s, a] y ejecuta shift, reduce, accept o reporta error. **Entrada:** tabla SLR + flujo de tokens. **Salida:** aceptación/rechazo y, opcionalmente, árbol sintáctico construido durante las reducciones.

8. Módulo de visualización

Genera artefactos para depuración: archivo Graphviz .dot del autómata LR(0) y volcado tabular de ACTION/GOTO. **Entrada:** autómata y tabla. **Salida:** imágenes y reportes.

3. Diagrama de interacciones

El flujo se lee de arriba hacia abajo: las entradas (archivos .yalp, .yalex y la fuente) son consumidas por los lectores; el análisis estático de la gramática produce el autómata y la tabla; finalmente, el motor del parser combina la tabla con los tokens del lexer. La visualización es una rama lateral, opcional pero muy útil para depurar conflictos.



4. Ejemplo: procesamiento manual de una cadena

Se utiliza la gramática clásica de expresiones aritméticas y se rastrea la cadena `id + id * id` a través de cada fase, indicando qué produce cada módulo.

Gramática de partida

```
E -> E + T | T
T -> T * F | F
F -> ( E ) | id
```

Fase 1 Lexer (YALex)

Entrada cruda: `"id + id * id"`. El DFA aplica máximo prefijo y produce el flujo de tokens, agregando el centinela `$` al final.

```
Salida: [ id, +, id, *, id, $ ]
```

Fase 2 Lectura del .yalp y aumentación

El lector YAPar transforma el archivo en una Grammar. Tras la aumentación queda $S' \rightarrow E$ y se numeran las producciones para usarlas en reducciones:

```
(0) S' -> E
(1) E  -> E + T
(2) E  -> T
(3) T  -> T * F
(4) T  -> F
(5) F  -> ( E )
(6) F  -> id
```

Terminales: { id, +, *, (,), \$ }. No terminales: { S', E, T, F }.

Fase 3 FIRST y FOLLOW

	FIRST	FOLLOW
S'	{ (, id }	{ \$ }
E	{ (, id }	{ +,), \$ }
T	{ (, id }	{ +, *,), \$ }
F	{ (, id }	{ +, *,), \$ }

Fase 4 Autómata LR(0): estados de ejemplo

Aplicando closure y goto sobre la gramática aumentada se obtienen 12 estados (numerados I0 a I11). A continuación se muestran tres representativos; el resto se construye por el mismo procedimiento:

```
I0 = closure({ S' -> .E })
    = { S' -> .E, E -> .E+T, E -> .T, T -> .T*F, T -> .F,
        F -> .(E), F -> .id }
```

```
I1 = goto(I0, E) = { S' -> E., E -> E.+T }
```

```
I5 = goto(I0, id) = { F -> id. }
```

Fase 5 Tabla SLR(1) (extracto relevante)

Convención: sN = shift al estado N, rk = reducir por la producción k, acc = aceptar.

Estado	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
5		r6	r6		r6	r6			
6	s5			s4				9	3
9		r1	s7		r1	r1			

Las columnas con fondo ámbar pertenecen a **ACTION**; las azules, a **GOTO**.

Fase 6 Ejecución del motor del parser

El motor opera con una pila que alterna estados y símbolos. Para la cadena $id + id * id \$$:

Pila	Entrada	Acción
0	$id + id * id \$$	shift 5
0 id 5	$+ id * id \$$	reduce 6 ($F \rightarrow id$)
0 F 3	$+ id * id \$$	reduce 4 ($T \rightarrow F$)
0 T 2	$+ id * id \$$	reduce 2 ($E \rightarrow T$)
0 E 1	$+ id * id \$$	shift 6
0 E 1 + 6	$id * id \$$	shift 5
0 E 1 + 6 id 5	$* id \$$	reduce 6 ($F \rightarrow id$)
0 E 1 + 6 F 3	$* id \$$	reduce 4 ($T \rightarrow F$)
0 E 1 + 6 T 9	$* id \$$	shift 7
0 E 1 + 6 T 9 * 7	$id \$$	shift 5
0 E 1 + 6 T 9 * 7 id 5	$\$$	reduce 6 ($F \rightarrow id$)
0 E 1 + 6 T 9 * 7 F 10	$\$$	reduce 3 ($T \rightarrow T * F$)
0 E 1 + 6 T 9	$\$$	reduce 1 ($E \rightarrow E + T$)
0 E 1	$\$$	accept

Resultado esperado: la cadena pertenece al lenguaje y, durante las reducciones, el motor emite el árbol de derivación correspondiente. Si en alguna celda ACTION[s, a] el contenido fuese vacío, se reportaría un error sintáctico con la posición original que vino del lexer.

5. Estructura de datos del autómata LR(0)

El autómata LR(0) se modela como un grafo dirigido cuyos vértices son *estados* (conjuntos de items) y cuyas aristas son transiciones etiquetadas por símbolos de la gramática. La clave del diseño es que **los items deben ser hashables** para que los estados puedan compararse por igualdad de conjuntos —ese es el mecanismo que evita duplicar estados durante la construcción canónica—, y que **las transiciones se almacenen por id de estado** para no romper la inmutabilidad de los conjuntos de items.

Definiciones (en pseudocódigo Python)

```
@dataclass(frozen=True)
class Symbol:
    name: str
    is_terminal: bool

@dataclass(frozen=True)
class Production:
    id: int          # número de producción
    lhs: Symbol      # no terminal
    rhs: tuple[Symbol, ...]

@dataclass(frozen=True)
class LR0Item:
    production: Production
    dot: int        # 0 .. len(rhs)
    def is_reduce(self) -> bool: return self.dot == len(self.production.rhs)
    def next_symbol(self) -> Symbol | None: ...

@dataclass
class State:
    id: int
    items: frozenset[LR0Item]      # cerradura completa
    transitions: dict[Symbol, int] # GOTO -> id de otro estado

class LR0Automaton:
    states: list[State]
    start: int = 0
    grammar: Grammar
    _index: dict[frozenset[LR0Item], int] # deduplicación
    def closure(self, items): ...
    def goto(self, state_id, symbol): ...
    def build(self): ... # colección canónica
```

Notas de diseño

Las producciones se identifican por un entero secuencial (*id*) porque las celdas de ACTION guardan ese número en lugar de la producción completa, lo que mantiene la tabla compacta y serializable a JSON. Los LR0Item son *frozen* para poder vivir dentro de un frozenset; los State, en cambio, se mantienen mutables solo durante la construcción —para ir agregando transiciones— y luego se tratan como inmutables. El diccionario auxiliar `_index: frozenset[LR0Item] -> int` es lo que garantiza que dos cerraduras iguales no generen estados duplicados, paso crítico para que el algoritmo termine.

Estructura asociada para la tabla SLR

Action = Shift(int) | Reduce(int) | Accept | Error

```
@dataclass
class SLRTable:
    action: dict[tuple[int, Symbol], Action]
    goto: dict[tuple[int, Symbol], int]
    conflicts: list[Conflict]
```

Esta separación de action y goto respeta la dicotomía clásica terminal/no terminal y permite que el motor del parser haga una sola búsqueda hash por paso. El campo `conflicts` registra colisiones detectadas durante el llenado, lo que da retroalimentación accionable al usuario que escribió la gramática en lugar de fallar silenciosamente.